

PORTFOLIO · PRODUCTION AI SYSTEMS · 2024–2026

Fourteen AI systems, one operating model: ship it, prove it, govern it.

A ~500-person global Employer-of-Record platform (hiring in 160+ countries) needed AI working across its entire go-to-market org. As GTM AI & Automation lead, I designed, shipped, and **operated** the systems below — used daily by 50+ non-technical reps and marketers, running unattended on live revenue pipelines.

14

production systems
shipped

50+

daily non-technical
users

~70%

enrichment cost cut
by design

\$250K

open deal traced to
its originating AI chat

All figures as evidenced in project records and verified against the live CRM — nothing modeled, nothing rounded up.

What repeats across every case study

- **Hybrid AI architecture.** LLMs for semantic mapping, drafting, classification; deterministic, unit-tested code wherever a decision must be auditable. Never "the AI decided."
- **Production discipline.** Dry-run harnesses, regression diffs, rollback plans, idempotent writes, failure-safe state, error routing.
- **Cost engineering.** AI and data spend measured, reported, and controlled by architecture — not by asking users to be careful.
- **Platform building.** A skills factory, a project Center of Excellence, and observability, so the system compounds instead of accumulating scripts.

CASE STUDIES

1. AI customer agent + revenue attribution

8. Enrichment cost governance (~70%)

https://claude.ai/code/artifact/ab6121da-592b-4752-8c34-32c64eb38a68?open_in_browser=1&via=user_open&org=068166dc-a8ad-4f23-827e-7740aa47abf8

2. AI compliance screening on a live pipeline

3. Four AI agents on every sales call

4. Pre-call briefing automation

5. Contract intake in under 3 minutes

6. Prospect pipeline for an SDR agency

7. Zero-UI prospecting engine
9. CRM import formatter

10. Failure-safe change detection

11. AI lead classification & routing

12. RFP response suite

13. Automation observability

14. The AI platform layer

01 LIVE IN PRODUCTION

AI Customer Agent Built End to End — Agent, Pipeline, Warehouse, Attribution

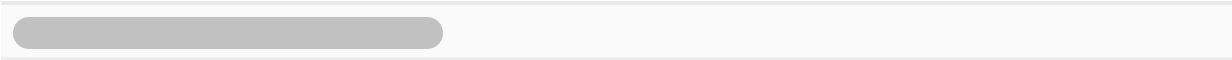
A company needed a production AI agent on its public site that qualifies visitors, feeds the CRM cleanly, and can prove its revenue impact. I built the entire application, start to finish.

① AGENT	Versioned conversation guidelines and guardrails (v4 → v6), knowledge base, qualification and handoff rules — live on the public pricing page since early 2026.
② PIPELINE	Live AI qualification, 4,100+ logged executions : closed-conversation webhook → contact resolution → hand-written transcript normalizer (visitor-vs-bot actor detection; email/phone/company extraction from free text) → LLM agent with structured-output parsing under governance rules I authored: only visitor messages count as intent, confidence < 0.65 forces Non-MQL, blank values never overwrite CRM data. MQLs get 15 properties stamped and a follow-up email drafted.
③ WAREHOUSE	Hybrid three-workflow reporting: real-time event collector, nightly reconciliation rebuilding source-of-truth metrics (including a "why didn't this conversation create a contact?" gap metric), and an hourly AI-credit watcher with 4-tier alerts and month-end spend forecasting.
④ ATTRIBUTION	Deal → contact → chat-session association walk with three honesty tiers (sourced / influenced / excluded), operationalized as a weekly stamper workflow writing attribution tier + originating conversation ID onto deals — report-only until sign-off.
RESULT	Verified against the live CRM (July 2026): a \$250K open deal sourced by the agent — chat one day, deal the next, traceable to the exact originating thread ID. ~\$520K agent-sourced deal value since launch;

~\$680K closed-won in 2026 on engaged contacts (influence tier, labeled as such); ~\$3.6M lifetime deal value across ~390 agent-engaged contacts.

Every layer protects the credibility of the number above it: dedup and legacy-exclusion keep contacts clean → confidence-gated MQL rules keep lifecycle honest → reconciliation catches what webhooks miss → attribution tiers separate "sourced" from "influenced" so no claim collapses under one follow-up question.

STACK HubSpot Customer Agent · n8n (LangChain agent nodes) · Azure



Demonstrates: full-stack AI product ownership · LLM classification with governance rails in a production CRM · attribution design honest enough for a board deck.

02 LIVE IN PRODUCTION

AI Compliance Screening on a Live Sales Pipeline

Deals were reaching contract stage before anyone checked regulatory eligibility — blockers surfaced at signature time, killing deals late.

CHALLENGE	Screen every deal entering the pipeline against a 30-sector compliance rulebook, automatically and auditably.
BUILT	A 46-node webhook + batch system: the LLM (GPT-4o) reads messy CRM data and call transcripts and maps them to canonical sectors — including pass-through detection (a staffing firm placing oil-and-gas workers is screened as oil and gas) — then a deterministic, unit-tested rules engine issues the verdict and writes tier, flags, and plain-English reasoning back to the CRM.
RESULT	All ~240 open deals screened; 103 deals wrongly stuck in "review required" correctly cleared while ineligible-deal detection doubled — every change traceable to a named rule. 238/238 production writes, zero errors.

The decision that matters: the rules live in code, not in the AI's head. Model confidence is a report field that can never change an outcome. Same deal in twice → same answer.

I sample went through 40 unit tests, a separate rulebook was applied to the entire live pipeline

Launch went through 10 unit tests, a report-only dry-run against the entire live pipeline (zero CRM writes), a before/after regression diff, and a parked rollback version.

STACK n8n cloud · Azure OpenAI GPT-4o · custom JS rules engine · I

Demonstrates: regulated-domain AI with full audit trails · LLM/deterministic hybrid design · zero-risk deployment methodology.

03 LIVE IN PRODUCTION

Four AI Agents on Every Sales Call

Reps left calls without follow-ups; country-specific facts — the heart of the product — were applied inconsistently.

BUILT

An always-on pipeline triggered by a conversation-intelligence webhook after every call: four AI agents produce a country-aware follow-up email, missed-questions analysis, competitor research, and a LinkedIn draft — plus an automatic CRM note. Zero rep action.

RESULT

Live on every sales call, any device, unattended.

The pivot worth telling: v1 used a vector-search index (RAG) for country facts. It broke, and permissions blocked a rebuild. Instead of patching, I deleted the retrieval layer: transcript → ISO-3 country codes → exact country files fetched from blob storage → injected into each agent's context. Deterministic, zero credentials to rotate, and a missing file degrades output instead of erroring. Verified end-to-end on a real three-country call — 116K characters of live context assembled. Knowing when *not* to use RAG is an architecture skill.

STACK n8n cloud · 4 LLM agents · Azure Blob (250+ country files)

Demonstrates: event-driven multi-agent pipelines · pragmatic RAG-vs-deterministic judgment · graceful degradation.

04 SHIPPED

Pre-Call Briefing Automation

The companion system: a workflow assembling AI briefings for reps ahead of scheduled meetings — shipped, like every build in this portfolio, with an executive one-pager, architecture diagram, and project brief. Together with N° 03 it brackets every sales call with AI.

STACK n8n · HubSpot · stakeholder deliverables (brief, diagram, one-pager)



Demonstrates: end-to-end ownership of the sales-call lifecycle · documentation as part of "done."

05 SHIPPED

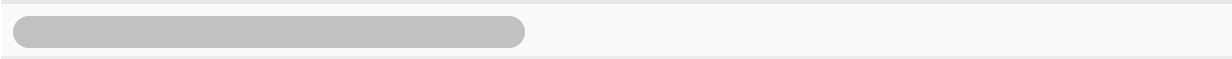
Contract Intake: CRM Deal to Legal Form in Under 3 Minutes

Sellers hand-copied deal data into Legal's e-signature intake form; errors stalled contracting.

BUILT	Seller pastes a deal URL → agent pulls deal/signer/terms and confirms in 3 steps → a webhook service validates and maps values to Legal's exact form schema → an AI browser agent pre-fills the form in the seller's own logged-in browser → seller attaches the contract and submits → Legal's signature workflow fires natively, untouched.
RESULT	Deal-to-filled-intake in under 3 minutes, with Legal's process completely unchanged.

The engineering story: two "obvious" API integrations were built and proven dead by live testing — documented with response codes and request IDs, then abandoned on evidence rather than exhaustion. The pivot to browser-session fill turned out *more* secure: no vendor credentials stored server-side, human review guaranteed, and the agent never clicks Submit by design.

STACK AI agent skill · CRM via MCP · n8n validation webhook · agent



Demonstrates: constraint-driven architecture · human-in-the-loop by design · vendor API forensics.

06 SHIPPED

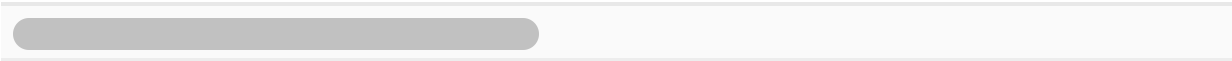
Prospect Data Pipeline Feeding an Outsourced SDR Agency

Feeding an outside calling agency stale or already-called contacts burns real money and brand equity.

BUILT	A Python pipeline (9 scripts + orchestrator): merges five enrichment exports, dedupes against every contact the agency ever called, backfills emails and mobile numbers via a sales-intelligence API, buckets contacts into ICP tiers, scrapes open international job postings from five ATS platforms as buying-intent signals, and emits dated weekly send plans.
RESULT	779 unique contacts across 623 companies with confirmed 0% overlap against ~2,900 already-called keys; 91% mobile coverage; tier-ranked 200-contact weekly plans.

Safeguards: triple-key dedup (name+company / email / LinkedIn) · per-role, per-company enrichment caps · title-based exclusion lists · phone dedup by last-10 digits · a full send log.

STACK Python · sales-intelligence API · enrichment-platform export



Demonstrates: data engineering for GTM · API enrichment at scale · cost-capped pipeline design.

07 SHIPPED

Zero-UI Prospecting Engine for Non-Technical Reps

CHALLENGE	The data-enrichment platform is powerful, but reps can't drive it — every list request bottlenecked on one person.
BUILT	A production AI skill: a rep uploads any CSV; the skill auto-detects from column signatures whether it's a company list or a contact list, routes to the right enrichment pipeline via REST API, and returns results by CRM sync, CSV, or email.
RESULT	Zero enrichment-platform UI for reps; the bottleneck removed.

Backed by a full RevOps agent charter: source-of-truth boundaries per system, an 18-code suppression taxonomy (customers, open deals, active sequences, bounces, do-not-contact...), a safe CRM update policy — fill blanks, never blind-overwrite, flag conflicts — per-run audit trails, and a secrets policy that forbids pasted credentials.

STACK AI skill · Clay REST API · HubSpot private-app auth · CSV v

Demonstrates: productizing internal tooling · compliance-first outbound · agent governance written like platform policy.

08 SHIPPED

Enrichment Cost Governance — ~70% Cost Cut by Architecture

A runaway enrichment spend happened on my watch — background workflows burned a large credit purchase in about a month, unmonitored. I own that failure; this is what I built so it can't recur.

The audit that followed examined every running workflow: builds averaging ~50 credits/row, duplicate phone purchases across providers, AI columns firing on non-target rows, zero pre-run cost visibility.

BUILT	A standard, cloneable enrichment table: a free ICP-fit gate before any paid column; AI research columns restricted to top-tier accounts; one decision-maker per company; one phone waterfall capped at three provider attempts; skip-logic on every populated field; a per-row cost-estimate column. Columns run cheap-to-expensive, each gated
-------	---

Column: Columns run cheap to expensive, each gated.

RESULT	Three predictable build profiles — Lean ~15 · Balanced ~21 · Full ~50 credits/row — plus monthly spend reports to the budget-owning VP.
--------	---

The principle: cost control enforced by structure, not by asking users to be careful.

STACK Clay (formulas, waterfalls, run-conditions, AI columns) · c



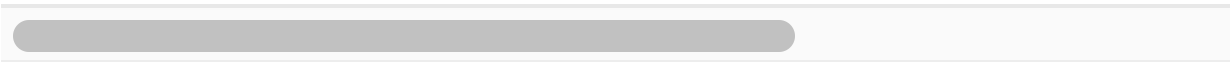
Demonstrates: FinOps for AI/data tooling · turning an audit into an enforced standard.

09 LIVE IN PRODUCTION

CRM Import Formatter — Killing Duplicates at the Source

CHALLENGE	Reps bulk-imported hand-made spreadsheets in inconsistent formats; the CRM couldn't dedupe — duplicates inflated forecasts and split activity history.
BUILT	An AI skill converting any spreadsheet (Apollo, Sales Navigator, ZoomInfo, partner lists) into the official import templates: auto-splits people from companies, dedupes pre-import, semantically maps messy industries to the CRM taxonomy enriched with NAICS codes, normalizes dropdowns and countries, flags — never silently drops — rejected rows, and optionally upserts directly with list creation.
RESULT	Live for a 50+ person org. Every import carries a batch ID used as the CRM import name — deals are attributable back to clean-data discipline. A run ledger reports adoption, volume, and quality KPIs.

STACK AI skill · Excel/CSV processing · HubSpot import API (upser



Demonstrates: root-cause data-quality engineering · adoption-aware design · measurable by default.

10

BUILD-READY · EXEC SIGN-OFF GATED

Failure-Safe Weekly Change Detection

The company's legal ability to operate in each country changes over time. Nobody caught the flips — Marketing kept paying for search ads in countries it could no longer serve.

BUILT	A weekly workflow pulling per-country readiness, normalizing to a strict boolean, diffing against last week's baseline, and emailing stakeholders only what flipped — a deliberately locked two-column report, per the executive requester's spec.
RESULT	6/6 unit checks pass; architecture routed through executive sign-off before activation — governance as a feature.

The reliability design: baseline writes sit downstream of delivery, so a failed pull or send can never corrupt last week's state; empty pulls fail loudly; new countries seed silently; a record disappearing from the feed doesn't fire a false alarm. State lives in native workflow data tables mirrored to Excel — honoring a hard "no extra cloud infrastructure" constraint from leadership.

STACK n8n (schedule trigger, Data Tables, code nodes) · internal (



Demonstrates: state management & idempotency · designing for failure modes first · engineering within stakeholder constraints.

11

DEPLOY-READY

AI Lead Classification & Routing

CHALLENGE	Partnership-form submissions mixed genuine partner leads with vendor spam; one person triaged everything by hand.
BUILT	Webhook → LLM classifies partner type (referral / alliance / integration vs. vendor pitch) → CRM properties and lifecycle stamped → a high-priority task routed to the partnerships owner with an AI-drafted response. Vendor pitches politely filtered: no task, no email.

Engineering notes: caught an API-version association bug pre-launch that would have created tasks with no contact attached — silently defeating the automation; hardened node references; wired the shared error handler; shipped with three test payloads defining expected behavior before go-live.

STACK n8n · Azure OpenAI · HubSpot API (properties, lifecycle, ta

Demonstrates: LLM classification in a production CRM path · defensive integration coding · test-first deployment.

12 PHASE 1 LIVE

RFP Response Suite — Days to Hours on Enterprise Questionnaires

BUILT	Three parts: a KB Builder parsing historical RFPs (PDF/DOCX/XLSX) and support tickets into atomic Q/A facts — classified, routed to owning teams, conflicts flagged; an Answer Retriever with honest HIGH/MEDIUM/NO-MATCH confidence, batch coverage summaries, and auto-created CRM tickets for every gap; and an RFP Responder agent that ingests a full RFP, extracts and categorizes every requirement across 8 categories, drafts compliant responses, and flags what needs human expertise.
RESULT	"We don't know" became an owned, ticketed work item instead of an invisible gap.

STACK AI agent skills · document parsing · semantic matching with

Demonstrates: knowledge-base architecture · honest-confidence retrieval · closing the loop from unknown to assigned.

13 IN DESIGN

Automation Observability —

SRE for the Workflow Stack

When a production workflow failed or quietly degraded, the team found out only when data stopped moving.

BUILDING	A read-only observability layer: point it at any workflow → every execution archived (idempotent upsert, every cited number traceable to a real execution ID) → three-layer analysis (failures + suggested fixes, performance, health) → auto-updating dashboard.
TARGETS	Time-to-diagnosis from 10–15 minutes to under 60 seconds; stalled workflows detected within 24 hours; ≥80% of failures shipped with an actionable fix suggestion.

Charter constraints by design: GET-only API access (can never modify a workflow), secrets never printed or committed, absent data recorded as null, alerts dry-run until explicitly enabled.

STACK Python · n8n public API (read-only) · dashboard layer

Demonstrates: SRE instincts for business automation · read-only-by-design safety · the step from building automations to operating a platform.

14 LIVE · ORG-WIDE

The AI Platform Layer — Skills Factory, CoE & Frameworks

One automation is a win. Fifty non-technical people using AI daily requires a system.

This is the layer everything above runs on:

METHOD	A published internal build framework covering prompt compilation, jobs-to-be-done planning, work classification, and provenance honesty (evidenced vs. inferred) — applied to every agent manifest in this portfolio.
COE ENGINE	Any knowledge, text files, links, processes, leadership, workbooks,...